cin >>x >> y;	This is used to extract two values. cin reads the first value and immediately assigns that to variable x; next, it reads the second value which is typed after a space and assigns that to y. Space is used as a separator for each input.

9.4.2 Output Operator

C++ provides << operator to perform output operation. The operator << is called the **"Stream insertion"** or **"put to"** operator. It is used to send the strings or values of the variables on its right to the object on its left. << is a binary operator.

The first operand is the pre-defined identifier cout (pronounced as C-Out) that identifies monitor as the standard output object. The second operand may be a constant, variable or an expression.

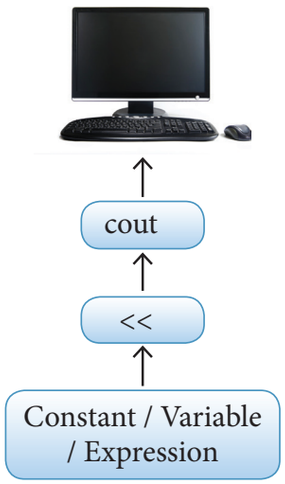


Figure 9.5 Working process of cout

To send more than one value at a time, << operator should be used for each constant/variable/expression. This is called **cascading of operator**.

Example:

cout << "Welcome";	Pre-defined object cout sends the given string "Welcome" to screen.
cout << "The sum = " << sum;	First, cout sends the string "The Sum = " to the screen and then sends the value of the variable sum; Usually, cout sends everything specified within double quotes or single quotes i.e., string or character constants, except non-graphic characters.
cout <<"\n The Area: " <<3.14*r*r;	First, cout sends everything specified within double quotes except \n to the screen, and then it evaluates the expression 3.14*r*r and sends the result to the screen. \n – is a non graphic character constant to feed a new line.
cout << a + b ;	cout sends the sum of a and b to the output console (monitor)

9.4.3. Cascading of I/O operators

The multiple use of input and output operators such as >> and << in a single statement is known as cascading of I/O operators.

Cascading cout:

```
int Num=20;
cout << "A=" << Num;
```

The Figure 9.6 is used to understand the working of Cascading cout statement

```
cout << "A=" << Num;
```

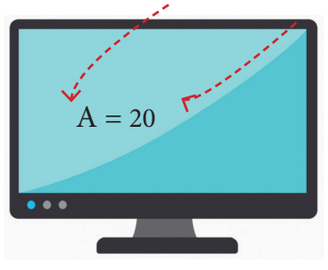


Figure 9.6 Cascading cout

Chapter 9 Page 115-151.indd 124 3/24/2020 9:21:04 AM